Why 90% of C++ Features Are Not Used in Aerospace

1. The Core Philosophy

In normal software development, the question is: 'What features can I add?' In aerospace software engineering, the question is the opposite: 'What features must I remove or avoid?' This is the central idea behind all aerospace coding standards.

The key principle, stated directly in the video:

> "It's not about what's in the code. It's about quantifying what you take out."

Every feature included in flight-critical software must be justified. Every feature excluded must also be justified — systematically, not arbitrarily. The goal is deterministic, predictable behavior at all times.

---

2. Why Predictability Matters More Than Convenience

In consumer software, a crash means restarting the app. In aerospace, a crash can mean loss of the vehicle and crew. This changes every engineering decision fundamentally.

The Garbage Collector Problem

Languages like Java, Python, and C# use a garbage collector (GC) — an automatic memory cleanup system that runs in the background. The problem is the GC runs whenever it decides to, pausing the program for milliseconds with zero programmer control.

At Mach 1 (~1,200 km/h), a few milliseconds of unexpected pause in flight control software is unacceptable. The system must respond in real time, every time, with guaranteed timing. This is why high-level languages with GC are banned in flight-critical code.

Determinism as a Hard Requirement

Aerospace code must be deterministic — given the same inputs, it must always produce the same outputs in the same time. No randomness. No timing uncertainty. No 'it usually works.' This requirement eliminates entire categories of C++ features.

3. The Ariane 5 Disaster — Case Study

Date: June 4, 1996 | Location: French Guiana | Cost: ~$500 million USD

What Happened

Europe's newest rocket, Ariane 5, launched successfully and flew perfectly for 36 seconds. Then a single unhandled software exception caused it to veer off course and self-destruct.

The Root Cause

The flight software attempted to convert a 64-bit floating point number (horizontal velocity) into a 16-bit integer. The number was too large to fit — an overflow occurred, an exception was thrown, and nothing in the system was designed to handle it.

The critical detail: this code was copied directly from Ariane 4, where it worked perfectly — because Ariane 4 never flew fast enough to generate a number that large. Ariane 5 was significantly faster. Nobody re-validated the assumption.

The Most Important Lesson

"The language did exactly what it was allowed to do."

This was not a compiler bug. The language behaved correctly. The engineers did not account for what the language was permitted to do in an edge case. This distinction is everything in aerospace engineering:

- → Don't just ask: does this feature work?
- → Ask: what is this feature allowed to do that I haven't anticipated?

Detection vs Recovery

The system detected the exception — but could not recover from it. Detection and recovery are two completely separate problems. In aerospace, both must be solved explicitly. Silent failures and unhandled errors are never acceptable.

4. The JSF C++ Coding Standard

The JSF (Joint Strike Fighter) C++ coding standard was developed for the F-35 Lightning II program by Lockheed Martin. It is a publicly available document containing ~220 Avoidance Rules (AV Rules) that define exactly which C++ features are permitted, restricted, or banned entirely.

Why C++ and Not Something Else?

The US Department of Defense fought a long internal battle — the 'language wars' — over which language to standardize on across all military branches. Ada (a language literally designed by the

DoD for safety-critical systems) was the main competitor. C++ won, but only as a heavily restricted subset. The JSF standard is that subset.

Key Banned / Restricted Features

JSF AV Rule 208: Exceptions shall not be used.

Why: Exceptions create unpredictable control flow — the program jumps to an unknown location when an error occurs. In a fighter jet, you must know exactly what your code does at every moment. The Ariane 5 disaster was partly an exception-handling failure. The F-35 team banned them entirely.

JSF AV Rule —: Dynamic memory allocation (new/delete) shall not be used after initialization.

Why: Heap allocation (new/delete/malloc/free) is non-deterministic — allocation time varies, fragmentation occurs, and memory leaks are possible. All memory must be allocated statically at startup so the system's memory footprint is fixed and known at all times.

JSF AV Rule —: Recursion shall not be used.

Why: Recursion has unpredictable stack depth. In an embedded system with fixed stack size, unbounded recursion causes stack overflow — a hard crash with no recovery. All loops must be iterative with provably bounded execution.

JSF AV Rule —: Multiple inheritance shall not be used.

Why: Multiple inheritance creates complex, hard-to-analyze object hierarchies and ambiguous method resolution. It makes code difficult to reason about statically — a requirement for certification.

Key Required Practices

- Always initialize variables — uninitialized variables are undefined behavior
- Use fixed-size data types (uint32_t, int16_t) — never assume int or long size
- No implicit type conversions — every conversion must be explicit and intentional
- No hidden side effects — functions must do exactly what they appear to do
- No silent failures — every error must be detectable and handled
- Deterministic behavior — same input must always produce same output in same time
- Bounded loops — every loop must have a provable maximum iteration count
- Clear naming, structured programming, and mandatory documentation

5. History of Software in Military Aircraft

F-4 Phantom (1960s–70s) — Zero Software

The F-4's 'computer' was literally a box of gears and cams — described by the DoD as working like a precision watch. The 'code' was the physical shape of metal components. No software, no bugs, no runtime surprises. What you saw was exactly what you got.

F-14 Tomcat — The First Real Microprocessor in an Aircraft

The F-14's variable sweep-wing design required continuous real-time mathematical optimization that was impossible for a human pilot to perform manually — especially during high-G combat maneuvers. This forced the development of an onboard microprocessor.

Garrett AiResearch (now Honeywell) built the Central Air Data Computer (CADC) for the F-14 — a fully programmable microprocessor created years before Intel's 4004 (1971), which textbooks incorrectly call the world's first. The Garrett chip was classified until 1998.

The F-14's code was approximately 2,500 lines, physically burned as microcode — fixed and immutable, solving one narrow problem: processing polynomial equations for air data (altitude, airspeed, Mach number).

The Fragmentation Era — Language Wars

As software entered military aircraft, each branch of the US military developed its own programming language:

- Air Force → JOVIAL (Jules's Own Version of the International Algorithmic Language)
- Navy → CMS-2 (used on the F-18 Hornet)
- Army → separate standards

Two languages, two hardware architectures, zero compatibility. No shared lessons. No interoperability. Exponentially growing costs as flight software grew from thousands to millions of lines. This fragmentation forced the Pentagon to standardize — eventually leading to the JSF C++ standard.

Software Growth Trajectory

- F-4 Phantom → 0 lines of code
- F-14 Tomcat → ~2,500 lines (burned microcode)
- F-18 Hornet → tens of thousands of lines
- F-22 Raptor → ~1.7 million lines
- F-35 Lightning II → ~8 million lines

This exponential growth is why strict coding standards became non-negotiable. Managing 8 million lines of safety-critical code across hundreds of contractors without a common rulebook would be catastrophic.

6. Modern Aircraft Are Computers That Happen to Fly

The F-35 is intentionally aerodynamically unstable — without software continuously making micro-corrections, it cannot maintain flight. The pilot does not directly control the aircraft. The pilot controls the software; the software controls the aircraft. This is called Fly-by-Wire.

This means software is now load-bearing infrastructure in aviation. It is not a convenience layer — it is the aircraft. The engineering rules that govern how that software is written are therefore life-or-death decisions, not style preferences.

---

7. Applicability to Embedded Systems & Firmware

The JSF standard is most relevant in avionics, missile guidance, space systems, and automotive safety (ISO 26262 / AUTOSAR). However its principles apply directly to any embedded systems work — IMUs, flight controllers, sensor fusion firmware, real-time control loops.

Practical Takeaways for Embedded/Firmware Engineers

- Avoid dynamic memory allocation in real-time control loops — use static allocation
- Always use fixed-width integer types (uint32_t, int16_t) — never assume type sizes
- Every loop must have a bounded maximum iteration count — no infinite loops without watchdogs
- Initialize every variable — uninitialized memory causes intermittent, unreproducible bugs
- Handle every error explicitly — no silent failures, no ignored return codes
- Keep functions small, single-purpose, and side-effect-free
- Use the JSF standard as a reference framework, not a rigid checklist, at learning stage